SL Unit Quiz Practice

CS 7641: Machine Learning

Solutions

1 Solutions to Question 1 – Decision Trees

Part 1. ID3 Algorithm Concepts

A) ID3 chooses the attribute with the lowest entropy.
False. ID3 selects the attribute with the *highest information gain*, which measures the reduction in entropy after the split, not simply the attribute with the lowest entropy.

B) Entropy is used to calculate information gain in ID3.
True. Information Gain (IG) is computed using entropy:

$$\mathrm{IG}(S, A) = H(S) - \sum_{v \in \mathrm{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

C) ID3 stops splitting if all remaining samples share the same class.
True. This is a base case of ID3; if a node contains only examples of one class, it becomes a leaf node.

D) ID3 prefers attributes with more levels because they reduce entropy.
False. ID3 may appear to favor multi-valued attributes due to overfitting, but it does not inherently prefer them unless the resulting information gain is higher. Gain ratio (used in C4.5) adjusts for this.

E) ID3 cannot directly handle numeric attributes without thresholds.
True. Numeric attributes must be discretized or handled using threshold-based splits in order to be used in ID3.

F) ID3 creates branches for each unique attribute value.
True. ID3 creates one branch for each value of a categorical attribute.

Part 2. Boosting and Decision Trees

A) Boosting adjusts weights to emphasize misclassified examples.
True. Misclassified examples have their weights increased so the next learner focuses more on them.

B) A weak learner must have an error rate strictly less than 0.5.
True. If a learner has error $\varepsilon \geq 0.5$, it is no better than random and is discarded in AdaBoost.

C) Boosting aggregates learner predictions with equal weight.
False. Each learner is assigned a weight:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

D) Boosting may overfit if too many learners are added.
True. Although boosting resists overfitting in many cases, it can overfit particularly noisy datasets.

E) Boosting is more effective with underfit base learners.
True. Boosting is designed to combine weak learners that perform just better than chance.

F) Boosting ignores examples that are correctly classified.
False. Correctly classified examples receive lower weights but are still included in the next round.

Part 3. ID3 Calculation

a) Entropy of GoSwimming:
4 examples are labeled Yes and 4 No.

$$H(S) = -\left(\frac{4}{8} \log_2 \frac{4}{8} + \frac{4}{8} \log_2 \frac{4}{8} \right) = -2 \cdot 0.5 \cdot \log_2 0.5 = 1$$

b) Information Gain for Weather:

- **Sunny (Examples 1, 2, 7)**: [No, No, Yes] Entropy $= -\left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) \approx 0.918$
- **Overcast (Examples 3, 6)**: [Yes, Yes] Entropy $= 0$
- **Rain (Examples 4, 5, 8)**: [Yes, No, No] Entropy = same as Sunny $= 0.918$

Weighted average entropy:

$$\frac{3}{8} \cdot 0.918 + \frac{2}{8} \cdot 0 + \frac{3}{8} \cdot 0.918 = 0.688$$

$$\text{IG(Weather)} = 1 - 0.688 = \boxed{0.312}$$

c) Information Gain for Pollution:

- **High (Examples 1, 2, 5, 8)**: All No, therefore Entropy = 0
- **Low (Examples 3, 4, 6, 7)**: All Yes, therefore Entropy = 0

$$\text{IG(Pollution)} = 1 - 0 = \boxed{1.0}$$

d) Attribute Chosen by ID3:
Pollution, since it provides the highest information gain.

Part 4. AdaBoost Step-by-Step

Let all sample weights be initially $w_i = \frac{1}{8}$.

a) Weighted Error ε_1:
Misclassified examples: Example 3 and 8

$$\varepsilon_1 = \frac{2}{8} = \boxed{0.25}$$

b) Compute α_1:

$$\alpha_1 = \frac{1}{2} \ln\left(\frac{1 - 0.25}{0.25}\right) = \frac{1}{2}\ln(3) \approx \boxed{0.5493}$$

c) Unnormalized Weights:

- **Misclassified**: $w' = \frac{1}{8} \cdot e^{0.5493} \approx 0.125 \cdot 1.732 \approx 0.217$
- **Correctly Classified**: $w' = \frac{1}{8} \cdot e^{-0.5493} \approx 0.125 \cdot 0.577 \approx 0.072$

d) Normalize Weights:

$$Z = 6 \cdot 0.072 + 2 \cdot 0.217 = 0.434 + 0.434 = 0.868$$

$$\text{Normalized (correct): } \frac{0.072}{0.868} \approx \boxed{0.083} \quad \text{Normalized (misclassified): } \frac{0.217}{0.868} \approx \boxed{0.250}$$

2 Solutions to Question 2 – Regression & Classification

Part 1: Polynomial Regression Behavior

- **A. True**. A degree-1 model (linear regression) cannot capture the evident curvature in the dataset and will underfit.
- **B. True**. A degree-7 model with 8 data points will interpolate the data exactly and result in zero training error.
- **C. False**. Degree-3 may not *always* outperform degree-2. Generalization performance depends on the data and overfitting risks.
- **D. True**. Increasing the polynomial degree increases model complexity, reducing bias but increasing variance.
- **E. False**. Higher-degree models generally increase variance due to overfitting.
- **F. True**. A degree-d polynomial has $d + 1$ parameters (including the constant term).

Part 2: Cross-Validation Interpretation

- **A. True**. Degree-1 has the simplest form, and hence likely has the lowest variance, although its bias may be high.
- **B. True**. Degree-7 has extremely high CV error, indicating overfitting to training data and poor generalization.
- **C. True**. Degree-3 has the lowest CV error in the table, suggesting it generalizes best among listed degrees.
- **D. False**. CV error does not always decrease; higher degrees can increase overfitting and error.
- **E. False**. Degree-7 is more sensitive to noise due to high variance and likely performs worse on noisy data.
- **F. True**. A good rule of thumb is to choose the model with the lowest CV error unless model simplicity is preferred.

Part 3. Degree-2 Polynomial Fit

- **a)** Design matrix X (columns: constant, x, x^2) and response y:

$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 4 & 16 \\ 1 & 5 & 25 \\ 1 & 6 & 36 \\ 1 & 7 & 49 \end{bmatrix}, \qquad y = \begin{bmatrix} 2 \\ 3 \\ 5 \\ 10 \\ 18 \\ 25 \\ 36 \\ 50 \end{bmatrix}.$$

Useful sums (with $x \in \{0, 1, \ldots, 7\}$):

$$\sum 1 = 8, \ \sum x = 28, \ \sum x^2 = 140, \ \sum x^3 = 784, \ \sum x^4 = 4676,$$

$$\sum y = 149, \quad \sum xy = 806, \quad \sum x^2 y = 4772.$$

- **b)** Normal equations for quadratic least squares (with $\beta = [a, b, c]^\top$):

$$(X^\top X)\beta = X^\top y, \quad X^\top X = \begin{bmatrix} 8 & 28 & 140 \\ 28 & 140 & 784 \\ 140 & 784 & 4676 \end{bmatrix}, \quad X^\top y = \begin{bmatrix} 149 \\ 806 \\ 4772 \end{bmatrix}.$$

  Solving,

$$\hat{\beta} = (X^\top X)^{-1} X^\top y = \begin{bmatrix} 2.125 \\ -0.43452381 \\ 1.02976190 \end{bmatrix}.$$

- **c)** Fitted model:

$$\hat{y}(x) \ = \ 2.125 \ - \ 0.43452381\, x \ + \ 1.02976190\, x^2.$$

- **d)** Prediction at $x = 8$:

$$\hat{y}(8) \ = \ 2.125 - 0.43452381 \cdot 8 + 1.02976190 \cdot 64 \ \approx \ 64.5536.$$

Part 4. Cross-Validation

- **a)** Split into 4 folds:
  - Fold 1: obs 1, 5
  - Fold 2: obs 2, 6
  - Fold 3: obs 3, 7
  - Fold 4: obs 4, 8
- **b)** For each fold, train on 6 examples, test on 2.
- **c)** Mean Squared Errors (approximate):

$$\text{Fold MSEs} = [1.89, \ 3.32, \ 5.21, \ 5.41]$$

- **d)** Average error:

$$\text{Avg MSE} = \frac{1.89 + 3.32 + 5.21 + 5.41}{4} = \boxed{3.96}$$

3 Question 3 - Neural Networks and Gradient Descent

Part 1. Gradient Descent Behavior

Assume we are minimizing a loss function $L(\theta)$ using gradient descent. Select all correct statements:

A. **True.** If the gradient is zero but the loss is not at a minimum, the algorithm has converged to a **saddle point**. Zero gradient implies a critical point, which could be a minimum, maximum, or saddle. If the loss is not minimal, it is a saddle point.

B. **False.** Gradient descent with a constant learning rate **does not always converge** for convex functions. If the learning rate is too large, the algorithm may overshoot and diverge.

C. **True.** A large learning rate can cause **oscillation or divergence** by repeatedly overshooting the minimum.

D. **True.** Using **momentum** can help the optimizer escape shallow minima or saddle points by incorporating past gradients into the update, adding inertia.

E. **False.** Gradient descent **does not guarantee** convergence to the global minimum for non-convex functions (e.g., neural networks), regardless of time.

F. **True.** The negative gradient points in the **direction of steepest descent** by definition.

Part 2. Neural Network Architecture

Consider a fully connected feedforward neural network with one hidden layer using ReLU activations. Select all true statements:

A. **False.** The ReLU function $\text{ReLU}(x) = \max(0, x)$ is **not differentiable at 0** due to a discontinuous derivative.

B. **False.** ReLU was introduced to **mitigate vanishing gradients**. While it can lead to dead neurons, it does not commonly suffer from vanishing gradients compared to sigmoid or tanh.

C. **True.** A network with a single hidden layer and sufficient **width** can approximate any continuous function, per the **Universal Approximation Theorem**.

D. **True.** If the input to a ReLU neuron remains negative, it **always outputs zero**, leading to **dead neurons** that stop learning.

E. **False.** Training deep ReLU networks still requires optimization algorithms like **gradient descent**.

F. **True.** ReLU activations output zero for negative inputs, leading to **sparse activations**.

Part 3. Weight Update Using Gradient Descent

Given:

$$x = 2, \quad w = 1.5, \quad y = 1, \quad \eta = 0.1, \quad \hat{y} = \sigma(wx), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

a) Compute predicted output:

$$z = wx = 1.5 \cdot 2 = 3, \quad \hat{y} = \frac{1}{1+e^{-3}} \approx 0.9526$$

b) Compute loss:

$$L = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})] = -\log(0.9526) \approx 0.0485$$

c) Compute gradient:

$$\frac{dL}{dw} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz} \cdot \frac{dz}{dw}$$

$$\frac{dL}{d\hat{y}} = -\frac{1}{\hat{y}} \approx -1.0497, \quad \frac{d\hat{y}}{dz} = \hat{y}(1-\hat{y}) \approx 0.0452, \quad \frac{dz}{dw} = x = 2$$

$$\frac{dL}{dw} \approx -1.0497 \cdot 0.0452 \cdot 2 \approx -0.0948$$

d) Update weight:

$$w_{\text{new}} = w - \eta \cdot \frac{dL}{dw} = 1.5 - 0.1 \cdot (-0.0948) = 1.5095$$

Part 4. Two-Layer NN Gradient Step

Given:

$$x = 1, \quad w_1 = 1, \quad w_2 = 2, \quad y = 3, \quad \eta = 0.05$$

$$\hat{y} = w_2 \cdot \text{ReLU}(w_1 x), \quad L = \frac{1}{2}(y - \hat{y})^2$$

a) Forward pass:

$$z_1 = w_1 x = 1, \quad a_1 = \text{ReLU}(1) = 1, \quad \hat{y} = w_2 \cdot a_1 = 2$$

b) Compute $\frac{dL}{dw_2}$:

$$\frac{dL}{d\hat{y}} = \hat{y} - y = -1, \quad \frac{d\hat{y}}{dw_2} = a_1 = 1 \Rightarrow \frac{dL}{dw_2} = -1$$

c) Compute $\frac{dL}{dw_1}$ via chain rule:

$$\frac{dL}{d\hat{y}} = -1, \quad \frac{d\hat{y}}{da_1} = w_2 = 2, \quad \frac{da_1}{dz_1} = 1, \quad \frac{dz_1}{dw_1} = x = 1$$

$$\frac{dL}{dw_1} = -1 \cdot 2 \cdot 1 \cdot 1 = -2$$

d) Gradient descent update:

$$w_2^{\text{new}} = 2 - 0.05 \cdot (-1) = 2.05, \quad w_1^{\text{new}} = 1 - 0.05 \cdot (-2) = 1.1$$

4 Question 4 - Instance Based Learning

Part 1. kNN Concepts

A. **True.** Scaling features rescales axes in the distance computation. If $x, z \in R^d$ and we scale feature j by $s_j > 0$, then $d(x, z)$ becomes $\|S(x - z)\|_2$ with $S = \mathrm{diag}(s_1, \ldots, s_d)$. Changing $\{s_j\}$ can change which points are nearest, hence changing the k-NN prediction.

B. **False.** Increasing k *typically increases bias and decreases variance.* It does not decrease both simultaneously in general.

C. **True.** Distance-weighted voting (e.g., $w_i = \frac{1}{(d(x_i,x)+\varepsilon)^p}$ or $w_i = \exp(-\gamma d(x_i, x)^2)$) down-weights far neighbors, making the prediction less sensitive to distant outliers.

D. **False.** Leave-one-out cross-validation for k-NN is standard and valid: the held-out point is excluded from the neighbor set, so there is no train/test contamination.

E. **False.** Manhattan (L_1) and Euclidean (L_2) distances can induce different neighbor orderings. Counterexample (query at the origin): for $a = (2, 0)$ and $b = (1.1, 1.1)$, L_1: $\|a\|_1 = 2 < \|b\|_1 = 2.2$ (so a is closer), L_2: $\|a\|_2 = 2 > \|b\|_2 = \sqrt{1.21 + 1.21} \approx 1.556$ (so b is closer).

Part 2. Scaling Matters

a) **Label = `No`.**

$$\begin{aligned}
d(q, A) &= \sqrt{(8-1)^2 + (22-16)^2} = \sqrt{85} \approx 9.220,\\
d(q, B) &= \sqrt{(8-2)^2 + (22-18)^2} = \sqrt{52} \approx 7.211,\\
d(q, C) &= \sqrt{(8-3)^2 + (22-28)^2} = \sqrt{61} \approx 7.810,\\
d(q, D) &= \sqrt{(8-12)^2 + (22-20)^2} = \sqrt{20} \approx 4.472,\\
d(q, E) &= \sqrt{(8-13)^2 + (22-30)^2} = \sqrt{89} \approx 9.434.
\end{aligned}$$

The $k = 3$ nearest are D (`No`), B (`No`), C (`Yes`). Majority vote: 2 `No` vs. 1 `Yes` $\Rightarrow$ `No`.

b) **Label = `No`.** First z-score each feature using the training data (population stats):

$$\mu_{\text{dist}} = \frac{1+2+3+12+13}{5} = 6.2, \quad \sigma_{\text{dist}} = \sqrt{\tfrac{1}{5}\sum(x-\mu)^2} \approx 5.192;$$

$$\mu_{\text{temp}} = \frac{16+18+28+20+30}{5} = 22.4, \quad \sigma_{\text{temp}} \approx 5.571.$$

The query in z-space is

$$q' = \left(\frac{8-6.2}{5.192}, \frac{22-22.4}{5.571}\right) \approx (0.347,\ -0.072).$$

Compute z-space distances (rounded):

$$\begin{aligned}
&d'(q', D') \approx 0.850, \quad d'(q', B') \approx 1.360, \quad d'(q', C') \approx 1.445,\\
&d'(q', A') \approx 1.725, \quad d'(q', E') \approx 1.729.
\end{aligned}$$

The $k = 3$ nearest remain D (`No`), B (`No`), C (`Yes`). Majority: `No`.

c) **Label = `No`.** With distance-weighted voting on the same three neighbors,

$$w_i = \frac{1}{d'(q', x_i') + 10^{-6}}, \quad w_D \approx 1.177,\ w_B \approx 0.735,\ w_C \approx 0.692.$$

Weight totals by class:

$$W_{\texttt{No}} = w_D + w_B \approx 1.912, \qquad W_{\texttt{Yes}} = w_C \approx 0.692.$$

Since $W_{\texttt{No}} > W_{\texttt{Yes}}$, the weighted vote is `No`.

Why scaling matters (takeaway). Before scaling, the `DistanceToTrail` feature (range ≈ 12) can dominate Euclidean distance relative to `AirTemp` (range ≈ 14 but differently distributed). z-scoring equalizes each feature to zero mean and unit variance, so each contributes comparably to distance. In this dataset, the nearest set is unchanged, but in general scaling can change neighbor identities and thus the prediction.

5 Question 5 - Ensemble B&B

Part 1. Bagging & Boosting Concepts

A. **True.** In bagging, each base learner is trained on a bootstrap sample of size n drawn *with replacement* from the n training points. Each model thus sees repeats and, on average, about $1-(1-\frac{1}{n})^n \approx 63.2\%$ unique examples.

B. **False.** Out-of-bag (OOB) error for a given tree is computed on the examples *not* included in that tree's bootstrap sample (the $\approx 36.8\%$ "left out" cases), providing an internal validation estimate.

C. **False.** AdaBoost *increases* the weights of *misclassified* examples and *decreases* the weights of correctly classified ones. A standard update is $w_i^{(t+1)} \propto w_i^{(t)} \exp\big(\alpha_t \mathbf{1}\{y_i \neq h_t(x_i)\}\big)$.

D. **True.** Gradient boosting adds learners fit to the *negative gradient* of the chosen loss with respect to current predictions: for residuals $r_i^{(t)} = -\left.\frac{\partial \ell(y_i, f(x_i))}{\partial f(x_i)}\right|_{f=f^{(t)}}$, the next learner h_t approximates $r^{(t)}$.

E. **False.** Random forests are *bagged* ensembles of (typically deep) decision trees with feature subsampling at each split; they are not built via boosting.

Part 2. Bagging & OOB Estimation

a) **OOB predictions by example (majority over OOB-only votes).**

- Ex 1: OOB from Tree 1 = Y, Tree 3 = N $\Rightarrow$ tie. *Tie rule (deterministic):* break toward `Y`. OOB pred = `Y`.
- Ex 2: OOB from Tree 2 = N, Tree 3 = N $\Rightarrow$ OOB pred = `N`.
- Ex 3: OOB from Tree 1 = Y, Tree 2 = Y $\Rightarrow$ OOB pred = `Y`.
- Ex 4: OOB from Tree 1 = Y, Tree 3 = N $\Rightarrow$ tie. Apply tie rule $\Rightarrow$ OOB pred = `Y`.
- Ex 5: OOB from Tree 2 = Y, Tree 3 = Y $\Rightarrow$ OOB pred = `Y`.
- Ex 6: OOB from Tree 1 = N, Tree 2 = N $\Rightarrow$ OOB pred = `N`.

b) **Overall OOB accuracy.**

Ex	True	OOB pred
1	Y	Y ()
2	N	N ()
3	Y	Y ()
4	N	Y ($\times$)
5	Y	Y ()
6	N	N ()

$$\Rightarrow \quad \text{Accuracy} = \frac{5}{6} \approx 0.833 \ (83.3\%).$$

Note: If you instead break ties toward `N`, Ex 1 would be wrong and Ex 4 correct, still 5/6 overall.

c) **Why OOB works (one sentence).** Each tree's OOB set acts like a built-in validation fold (about 36.8% of cases are left out per tree), so aggregating OOB predictions across trees provides an approximately unbiased, cross-validation–like estimate of generalization without holding out a separate set.

6 Question 6 - Kernel Methods

Part 1. Kernel SVM Fundamentals

A. **True.** Any positive semidefinite (PSD) kernel K induces an inner product in a (possibly infinite-dimensional) feature space $\mathcal{H}$: there exists a feature map ϕ such that $K(x, z) = \langle \phi(x), \phi(z) \rangle_{\mathcal{H}}$ (RKHS / Moore–Aronszajn).

B. **False.** Raising the polynomial degree generally *increases* model capacity (lower bias, higher variance), which can *increase* overfitting rather than reduce it.

C. **True.** The kernel trick evaluates $K(x, z)$ directly so the SVM decision $f(x) = \text{sign}(\sum_i \alpha_i y_i K(x, x_i) + b)$ never needs the explicit coordinates of $\phi(x)$.

D. **False.** The RBF kernel corresponds to an *infinite*-dimensional feature map; the number of support vectors affects the *decision expansion*, not the feature-space dimensionality.

E. **False.** Kernel SVM training scales primarily with the number of samples n (Gram matrix of size $n \times n$; memory $\mathcal{O}(n^2)$ and solver time often superlinear in n), not just with feature dimension.

Part 2. Polynomial Kernal SVM Decision

a) **Kernel evaluations at** $x^* = (1, 1)$:

$$k(x_1, x^*) = (\langle (1, 0), (1, 1) \rangle + 1)^2 = (1 + 1)^2 = 4,$$

$$k(x_2, x^*) = (\langle (0, 1), (1, 1) \rangle + 1)^2 = (1 + 1)^2 = 4.$$

b) **Decision value and class:**

$$\begin{aligned} f(x^*) &= \alpha_1 y_1 \, k(x_1, x^*) + \alpha_2 y_2 \, k(x_2, x^*) + b \\ &= 0.6 \cdot (+1) \cdot 4 \; + \; 0.4 \cdot (-1) \cdot 4 \; - \; 0.2 = 2.4 - 1.6 - 0.2 = 0.6. \end{aligned}$$

Thus $\hat{y} = \text{sign}(f(x^*)) = +1$.

c) **Gram matrix** $K_{ij} = k(x_i, x_j)$ for $i, j \in \{1, 2\}$:

$$\begin{aligned} k(x_1, x_1) &= (\langle (1, 0), (1, 0) \rangle + 1)^2 = (1 + 1)^2 = 4, \\ k(x_1, x_2) &= (\langle (1, 0), (0, 1) \rangle + 1)^2 = (0 + 1)^2 = 1, \\ k(x_2, x_1) &= k(x_1, x_2) = 1, \\ k(x_2, x_2) &= (\langle (0, 1), (0, 1) \rangle + 1)^2 = (1 + 1)^2 = 4. \end{aligned} \qquad \Rightarrow \qquad K = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}.$$

7 Question 7 - PAC Learning

A hypothesis class $\mathcal{H}$ is said to be **PAC (Probably Approximately Correct)** learnable if there exists a learning algorithm that, for any $\epsilon, \delta > 0$, can find a hypothesis $h \in \mathcal{H}$ such that:

$$\Pr[\text{error}_D(h) > \epsilon] \leq \delta$$

with sample complexity:

$$m \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\left(\frac{1}{\delta}\right)\right)$$

Part 1. PAC Concepts

A. **False.** PAC learning does **not guarantee zero training error**. It guarantees that with high probability, the generalization error is small. A hypothesis might have nonzero training error and still satisfy the PAC criterion.

B. **True.** As $|\mathcal{H}|$ increases, the term $\ln|\mathcal{H}|$ increases, which in turn increases the required number of training examples to maintain a low generalization error. This reflects the **complexity penalty** for large hypothesis spaces.

C. **False.** Infinite hypothesis classes **can be PAC learnable** if they have finite VC dimension. For instance, linear classifiers in R^d form an infinite hypothesis class but are still PAC learnable due to finite VC dimension.

D. **True.** PAC bounds **trade off accuracy (ϵ) and confidence (δ)** through the sample complexity formula. Achieving higher confidence or lower error both require more samples.

E. **True.** The PAC sample complexity grows **linearly with** $\frac{1}{\epsilon}$ and **logarithmically with** $\frac{1}{\delta}$. This means decreasing the error tolerance has a stronger effect on sample requirements than increasing the confidence.

F. **False.** A **consistent learner** returns hypotheses that correctly classify the training data, but this does not guarantee generalization to the true distribution. To ensure low generalization error with high probability, the number of samples must satisfy PAC bounds.

Part 2. PAC Sample Complexity

Given:

$$\epsilon = 0.05, \quad \delta = 0.01$$

$$m \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\left(\frac{1}{\delta}\right)\right) = 20\left(\ln|\mathcal{H}| + \ln(100)\right) = 20\left(\ln|\mathcal{H}| + 4.6052\right)$$

a) $|\mathcal{H}| = 100$:

$$\ln(100) = 4.6052 \quad \Rightarrow m \geq 20(4.6052 + 4.6052) = 20 \cdot 9.2104 \approx 184.2$$

So, $m \geq 185$ samples.

b) $|\mathcal{H}| = 10,000$:

$$\ln(10,000) = 9.2104 \quad \Rightarrow m \geq 20(9.2104 + 4.6052) = 20 \cdot 13.8156 \approx 276.3$$

So, $m \geq 277$ samples.

c) $|\mathcal{H}| = 1,000,000$:

$$\ln(1,000,000) = 13.8155 \quad \Rightarrow m \geq 20(13.8155 + 4.6052) = 20 \cdot 18.4207 \approx 368.4$$

So, $m \geq 369$ samples.

8 Question 8 - VC Dimensions

Part 1. VC Dimension Fundamentals

A. **False.** If the VC-dimension is 5, then there *exists* at least one set of 5 points that $\mathcal{H}$ can shatter; it does *not* mean $\mathcal{H}$ can shatter *any* set of 5 points. Moreover, no set of 6 points can be shattered by $\mathcal{H}$.

B. **False.** VC dimension depends on the **expressiveness of the hypothesis class**, not just the number of features. For example, polynomial degree, decision tree depth, or other representational limits affect VC-dimension.

C. **True.** VC dimension is often viewed as a measure of the **capacity or complexity** of a hypothesis class—it gives an upper bound on how expressive the class is in terms of dichotomies it can realize.

D. **True.** Higher VC dimension means the hypothesis class can fit more complex patterns, and thus requires **more data** to avoid overfitting and to ensure good generalization.

E. **False.** VC dimension is a property of the **hypothesis class only**, not of the data distribution. It is a worst-case measure independent of any specific distribution.

F. **True.** A hypothesis class with **finite VC dimension is a necessary condition for PAC learnability** (under standard assumptions like realizability).

Part 2. VC Dimension Understanding

A. **True.** Thresholds on R (classifiers of the form $\text{sign}(x - t)$ or an equivalent one–split rule) can shatter any single point but cannot realize all 2^2 labelings of two ordered points (e.g., $+, -$ is impossible), hence VCdim $= 1$.

B. **True.** Axis-aligned rectangles in R^2 can shatter 4 points in general position (one in each "corner" arrangement) but cannot shatter every set of 5 points; thus VCdim $= 4$.

C. **True.** The class of constant binary functions contains two hypotheses (all-0 and all-1). Any single point can be labeled either way by choosing the appropriate constant, but two points cannot realize mixed labels; therefore VCdim $= 1$.

D. **True.** For 1-NN with no restriction on prototype placement, any finite set with arbitrary labels can be memorized by taking the training set to be the sample itself (ties broken suitably). Hence the class can shatter sets of any finite size, giving *infinite* VC dimension.

E. **False.** A decision stump in R is exactly a one-threshold classifier (optionally with flipped polarity), whose VC dimension is 1, not 2.

F. **False.** VC dimension is a property of a *hypothesis class* of binary functions (equivalently, a family of measurable sets), independent of whether a learning *algorithm* is deterministic or randomized. It is well-defined whenever the class is well-defined.

Part 3. Axis-Aligned Rectangles in R^2

a) **VC-dim**$(\mathcal{H}) = 4$**.** *Lower bound:* There exists a 4-point set in R^2 that is shattered by axis-aligned rectangles (see note in (b) below). *Upper bound:* No 5-point set can be

shattered. For any 5 points, pick one point p that lies inside (or on the boundary of) the axis-aligned bounding box of the other four.[1] Any rectangle that contains those four must also contain p, so the labeling "the four are positive, p is negative" is unrealizable. Hence VC-dim ≤ 4, and with the lower bound this gives exactly 4.

b) **Important correction about the concrete set P.** The specific set $P = \{(0,0),(0,1),(1,0),(1,1)\}$ *cannot* be shattered by axis-aligned rectangles. Counterexample labeling: mark $(0,0)$ and $(1,1)$ as positive and $(0,1)$ and $(1,0)$ as negative. Any axis-aligned rectangle that contains both $(0,0)$ and $(1,1)$ must contain the entire unit square $[0,1] \times [0,1]$, and thus also contains $(0,1)$ and $(1,0)$—contradiction. Therefore, not *every* dichotomy on P is realizable.

How to exhibit a 4-point set that ***is*** *shattered:* Take $Q = \{(0,0),(1,3),(2,1),(3,4)\}$ (no two share an x or y and the points form a "staircase"). For any labeling, define the rectangle as $[\min_{i\in S} x_i,\ \max_{i\in S} x_i] \times [\min_{i\in S} y_i,\ \max_{i\in S} y_i]$, where S is the set of positives (and take an empty rectangle off to the side if $S =$). Because of the interleaved y-ordering versus x-ordering, this rectangle includes exactly the positives and excludes the negatives; running through the 2^4 labelings verifies shattering. (Many other 4-point configurations also work; the unit-square corners do not.)

c) **A 5-point labeling that no rectangle can realize (using the set $P \cup \{(2,2)\}$).** Label the four points in P as *positive* and label $(2,2)$ as *negative*. Any rectangle that contains all four points of P must contain the bounding box $[0,1] \times [0,1]$, which does *not* force inclusion of $(2,2)$—so this one *is* realizable. To produce a *non-realizable* labeling, instead choose one of the five points that lies inside the bounding box of the other four and label it *negative* while labeling those four *positive*. For example, with $\{(0,0),(0,1),(1,0),(1,1),(2,2)\}$, the point $(0,0)$ lies in the bounding box of the other four (which is $[0,2] \times [0,2]$). Label $(0,0)$ as negative and the other four as positive. Any axis-aligned rectangle that contains the other four must include $[0,2] \times [0,2]$ and hence also includes $(0,0)$, a contradiction. Thus this 5-point labeling cannot be implemented by any axis-aligned rectangle.

Part 4. VC Dimension of Geometric Classifiers

a) **VC-dim = 3** for all (closed) circles not centered at the origin in R^2 (i.e., all disks with center $c \neq (0,0)$). *Reason.* Disks in R^2 have VC dimension 3: any 3 points in general position can be shattered; but there exists a 4-point configuration (e.g., the vertices of a square) for which the labeling "two opposite positives, two opposite negatives" cannot be realized by any disk (the minimum circle through opposite vertices contains the other two). Excluding the measure–zero subset of circles centered exactly at the origin does not reduce the class's shattering power, so the VC dimension remains 3.

b) **VC-dim = 4** for all annuli (regions $\{x : r_{\min} \leq \|x - c\|_2 \leq r_{\max}\}$ with a common center c and $0 \leq r_{\min} < r_{\max}$), not centered at the origin. *Reason. Lower bound:* One can place 4 points so that, by choosing an appropriate center c and inner/outer radii $(r_{\min}, r_{\max})$, every 0/1 labeling is realized (think of arranging points so their distances to a movable center can be ordered to match any labeled subset, then selecting an interval

[1] Such a point always exists by a pigeonhole/orthant argument.

in that radial order). *Upper bound:* Annuli are intersections of an outer disk with the complement of an inner disk *sharing the same center.* This radial band structure prevents shattering of certain 5-point configurations (e.g., a regular pentagon with alternating labels requires two disjoint radial bands), so no 5-point set can be shattered. Hence the exact VC dimension is 4.

c) **VC-dim = 5** for a tesseract (axis-aligned hypercube) classifier in R^4 (positives are points inside some translated, uniformly scaled L_∞-ball). *Reason.* Axis-aligned hyperrectangles in R^d have VC dimension $2d$. Imposing the *equal side-length* constraint (hypercube) couples each dimension's lower/upper thresholds into a single scale parameter, reducing capacity to $d + 1$ (translation: d degrees; scale: 1). In particular, for $d = 4$ the class shatters some 5-point set (e.g., a carefully placed set involving the origin and the 4 coordinate directions), but cannot shatter any 6-point set under the cube constraint, yielding VC dimension $4 + 1 = 5$.